

Program 3

Title: **Implement Abstract Factory pattern in java.**

```
// Vehicle Interface
```

```
interface Vehicle {  
    void start();  
    void stop();  
}
```

```
// Concrete Classes for Car Brands
```

```
class Honda implements Vehicle {  
    public void start() {  
        System.out.println("Honda Car is starting");  
    }  
  
    public void stop() {  
        System.out.println("Honda Car is stopping");  
    }  
}
```

```
class Toyota implements Vehicle {  
    public void start() {  
        System.out.println("Toyota Car is starting");  
    }  
  
    public void stop() {  
        System.out.println("Toyota Car is stopping");  
    }  
}
```

```
}
```

```
class BMW implements Vehicle {
```

```
    public void start() {
```

```
        System.out.println("BMW Car is starting");
```

```
    }
```

```
    public void stop() {
```

```
        System.out.println("BMW Car is stopping");
```

```
    }
```

```
}
```

```
// Abstract Factory Interface
```

```
interface VehicleFactory {
```

```
    Vehicle createVehicle();
```

```
}
```

```
// Concrete Factories
```

```
class HondaFactory implements VehicleFactory {
```

```
    public Vehicle createVehicle() {
```

```
        return new Honda();
```

```
    }
```

```
}
```

```
class ToyotaFactory implements VehicleFactory {
```

```
    public Vehicle createVehicle() {
```

```
        return new Toyota();
```

```
    }
```

```
}
```

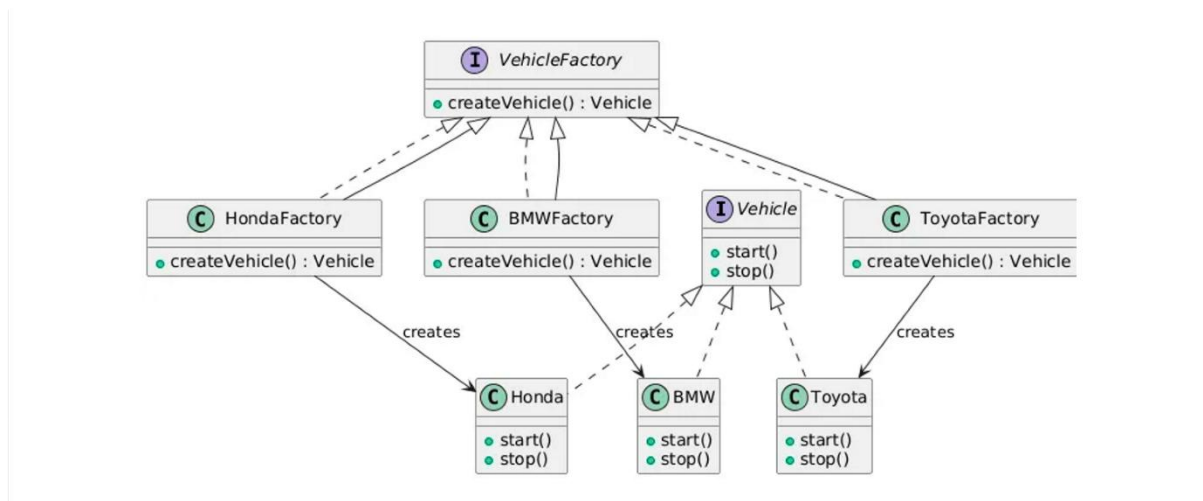
```
class BMWFactory implements VehicleFactory {  
    public Vehicle createVehicle() {  
        return new BMW();  
    }  
}
```

// Main Class

```
public class Main {  
    public static void main(String[] args) {  
  
        VehicleFactory hondaFactory = new HondaFactory();  
        Vehicle honda = hondaFactory.createVehicle();  
        honda.start();  
        honda.stop();  
  
        VehicleFactory toyotaFactory = new ToyotaFactory();  
        Vehicle toyota = toyotaFactory.createVehicle();  
        toyota.start();  
        toyota.stop();  
  
        VehicleFactory bmwFactory = new BMWFactory();  
        Vehicle bmw = bmwFactory.createVehicle();  
        bmw.start();  
        bmw.stop();  
    }  
}
```

Output

Honda Car is starting
Honda Car is stopping
Toyota Car is starting
Toyota Car is stopping
BMW Car is starting
BMW Car is stopping



Advantages of the Abstract Factory Pattern

- Easier to Extend:

Adding new car brands (or any other related products) is as simple as adding a new concrete factory. You don't need to touch the client code or the existing factories.

- Cleaner and More Maintainable:

Instead of modifying a large `createVehicle` method every time you need to add a new product, you encapsulate the logic in separate factory classes, making the system easier to maintain and extend.

- Consistency:

All objects in a family are created in a consistent manner. Whether it's creating vehicles or furniture

Real-life Use Cases and Examples

Here are a few places where the Abstract Factory pattern is commonly used:

- **Cross-Platform UI Libraries:**

If you're developing a cross-platform application, you can use an Abstract Factory to create platform-specific UI elements (buttons, windows, textboxes) for Windows, Mac, or Android, ensuring consistency across platforms.

- **Database Connections:**

In a multi-database system, you can use an Abstract Factory to create database connections for different databases like MySQL, PostgreSQL, or MongoDB.

Conclusion

The Abstract Factory Design Pattern provides a powerful way to manage the creation of related objects without specifying their concrete classes. It makes your system more scalable, maintainable, and easier to extend. Unlike the Factory Method, which works well for single products, the Abstract Factory is designed to handle families of related products with ease, making it an essential pattern in complex systems.